

Topic: Introduction to Strings in Java

Java Strings

In Java, a string is a sequence of characters. For example, "hello" is a string containing a sequence of characters 'h', 'e', 'l', 'l', and 'o'.

We use **double quotes** to represent a string in Java. For example,

```
// create a string
String type = "Java programming";
```

Here, we have created a string variable named *type*. The variable is initialized with the string Java Programming.

Different ways to create Strings in Java

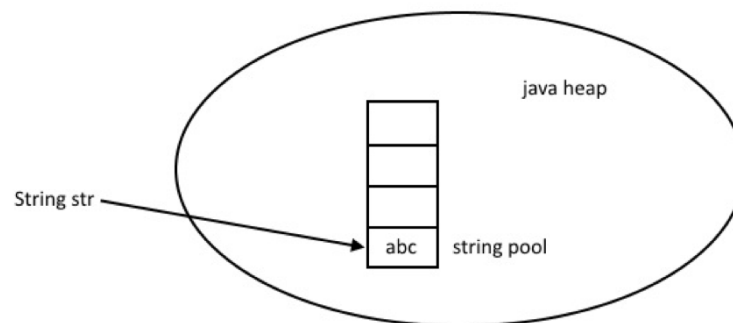
In Java, a string is an object that represents a sequence of characters or char values. The *java.lang.String* class is used to create a Java string object.

There are two ways to create a String object:

1. Using string literal

This is the most common way of creating string. In this case a string literal is enclosed with double quotes.

```
String str = "abc";
```



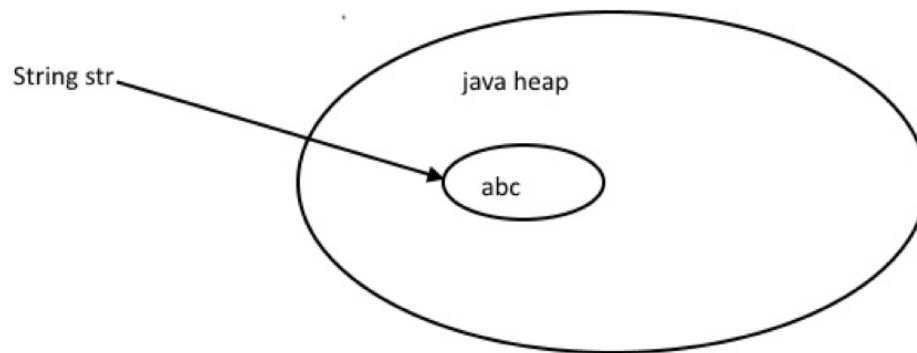
- When we create a String using double quotes, JVM looks in the String pool to find if any other String is stored with same value.
- If found, it just returns the reference to that String object else it creates a new String object with given value and stores it in the String pool.

2. Using new keyword

We can create String object using new operator, just like any normal java class. There are several constructors available in String class to get String from char array, byte array, StringBuffer and StringBuilder.

```
String str = new String("abc");

char[] a = {'a', 'b', 'c'};
String str2 = new String(a);
```



Topic: Methods for Extracting Characters from Strings

1. charAt()

The `charAt()` method returns the character at a specified index in a string. The index is zero-based, meaning the first character is at index 0.

Syntax:

```
public char charAt(int index)
```

Example:

```
public class CharAtExample {  
    public static void main(String[] args) {  
        String str = "Hello, World!";  
        char ch = str.charAt(7);  
  
        System.out.println("Character at index 7: " + ch);  
    }  
}
```

Output:

Character at index 7: W

2. getChars()

The `getChars()` method copies characters from a specified segment of a string into a destination character array.

Syntax:

```
public void getChars(int srcBegin, int srcEnd, char[] dst, int  
dstBegin)
```

- `srcBegin`: The starting index (inclusive).

- srcEnd: The ending index (exclusive).
- dst: The destination array.
- dstBegin: The starting index in the destination array.

Example:

```
public class GetCharsExample {
    public static void main(String[] args) {
        String str = "Hello, World!";
        char[] dst = new char[5];

        str.getChars(7, 12, dst, 0);

        System.out.println("Extracted characters: " + new
String(dst));
    }
}
```

Output:

Extracted characters: World

3. toCharArray()

The toCharArray() method converts the string to a new character array.

Syntax:

```
public char[] toCharArray()
```

Example:

```
public class ToCharArrayExample {
    public static void main(String[] args) {
        String str = "Hello";
        char[] charArray = str.toCharArray();

        System.out.println("Character array: " +
Arrays.toString(charArray));
    }
}
```

Output:

Character array: [H, e, l, l, o]

4. substring()

substring() method extracts a part of the string (substring) and returns it.

Syntax

```
public String substring(int startIndex, int endIndex)
```

In case of String:

- **startIndex**: inclusive
- **endIndex**: exclusive

Example:

```
public class TestSubstring{
    public static void main(String args[]){
        String s="SachinTendulkar";
        System.out.println("Original String: " + s);

        System.out.println("Substring starting from index 6: " +s
        .substring(6)); //Tendulkar

        System.out.println("Substring starting from index 0 to 6:
        "+s.substring(0,6)); //Sachin
    }
}
```

Output:

```
Original String: SachinTendulkar
Substring starting from index 6: Tendulkar
Substring starting from index 0 to 6: Sachin
```

Topic: Methods for comparing strings

In Java, comparing strings can be done in several ways, depending on what exactly you want to achieve. Here's a breakdown of the common methods for string comparison:

1. Using equals () Method

The equals () method is used to compare the contents of two strings for equality. It returns true if the strings are identical, and false otherwise.

```
String str1 = "hello";
String str2 = "hello";
String str3 = "world";

boolean isEqual1 = str1.equals(str2); // true
boolean isEqual2 = str1.equals(str3); // false
```

Note: equals () is case-sensitive. For a case-insensitive comparison, use equalsIgnoreCase ()

```
String str1 = "Hello";
String str2 = "hello";

boolean isEqual = str1.equalsIgnoreCase(str2); // true
```

2. Using compareTo () Method

The compareTo () method is used to compare two strings lexicographically. It returns:

- 0 if the strings are equal,
- A negative integer if the first string is lexicographically less than the second string,
- A positive integer if the first string is lexicographically greater than the second string.

```
String str1 = "apple";
String str2 = "banana";

int result = str1.compareTo(str2); // Negative value
```

Note: compareTo () is case-sensitive. For a case-insensitive comparison, use compareToIgnoreCase ().

```
String str1 = "Apple";
String str2 = "apple";

int result = str1.compareToIgnoreCase(str2); // 0
```

3. Using == Operator

The == operator checks if two string references point to the same object in memory. It does not compare the contents of the strings.

```
String str1 = new String("hello");
String str2 = new String("hello");

boolean b = (str1 == str2); // false
```

In this example, str1 and str2 have the same content but are different objects, so == returns false. To check for content equality, use equals ().

4. Using matches () Method

The matches () method is used for pattern matching with regular expressions.

```
String str = "hello123";
boolean b = str.matches("hello\\d+"); // true
```

5. Using `startsWith()` and `endsWith()` Methods

These methods check if a string starts or ends with a specified prefix or suffix, respectively.

```
String str = "hello world";

boolean b1 = str.startsWith("hello"); // true
boolean b2 = str.endsWith("world"); // true
```

6. Using `contains()` Method

The `contains()` method checks if a string contains a specified sequence of characters.

```
String str = "hello world";

boolean b1 = str.contains("hello"); // true
boolean b2 = str.contains("java"); // false
```

Topic: Methods to modify Strings

1. `concat()`

Concatenates the specified string to the end of the current string.

```
String str1 = "Hello";
String str2 = " World";
String result = str1.concat(str2); // "Hello World"
```

2. `replace()`

Replaces all occurrences of a specified character or substring with another character or substring.

```
String str = "Hello World";
String result = str.replace("World", "Java"); // "Hello Java"
```

3. `replaceAll()`

Replaces each substring of the string that matches the given regular expression with the given replacement.

```
String str = "Hello 123 World";
```

```
String result = str.replaceAll("\\d+", "#"); // "Hello #
World"
```

4. replaceFirst()

Replaces the first substring that matches the given regular expression with the given replacement.

```
String str = "Hello 123 World 456";
String result = str.replaceFirst("\\d+", "#"); // "Hello #
World 456"
```

5. toLowerCase()

Converts all characters in the string to lowercase.

```
String str = "Hello World";
String result = str.toLowerCase(); // "hello world"
```

6. toUpperCase()

Converts all characters in the string to uppercase.

```
String str = "Hello World";
String result = str.toUpperCase(); // "HELLO WORLD"
```

7. trim()

Removes leading and trailing whitespace from the string.

```
String str = "  Hello World  ";
String result = str.trim(); // "Hello World"
```

8. strip()

Removes leading and trailing whitespace using Unicode-aware whitespace characters.

```
String str = "  Hello World  ";
String result = str.strip(); // "Hello World"
```

9. stripLeading() and stripTrailing()

Remove leading or trailing whitespace characters.

```
String str = "  Hello World  ";
String resultLeading = str.stripLeading(); // "Hello World  "
```

```
String resultTrailing = str.stripTrailing(); // "  Hello
World"
```

10. format()

Formats the string using the specified format string and arguments, similar to `printf` in C.

```
String formatted = String.format("Hello %s, you have %d new
messages", "Mythri", 5);
// "Hello Mythri, you have 5 new messages"
```

11. valueOf()

Converts different types of values (e.g., integers, objects) to their string representation.

```
int number = 123;
String str = String.valueOf(number); // "123"
```

12. split()

Splits the string around matches of the given regular expression.

```
String str = "apple,banana,orange";
String[] result = str.split(","); // ["apple", "banana",
"orange"]
```

13. join()

Joins an array of strings into a single string with a specified delimiter.

```
String[] fruits = {"apple", "banana", "orange"};
String result = String.join(", ", fruits); // "apple, banana,
orange"
```

Topic: Methods for searching in strings

In Java, there are several methods available for searching within strings. These methods are part of the `String` class and can be used to find characters, substrings, or patterns within a string.

Commonly used methods for searching in strings:

1. indexOf()

Finds the index of the first occurrence of a specified character or substring. Returns `-1` if the character or substring is not found.

```
String str = "Hello, world!";
```

```
int index = str.indexOf("world");
if (index != -1) {
    System.out.println("Found at index: " + index);
} else {
    System.out.println("Not found");
}
```

Output:

Found at index: 7

2. lastIndexOf()

Similar to `indexOf()`, but returns the index of the last occurrence of the specified substring or character.

```
String str = "Hello, world! world!";
int index = str.lastIndexOf("world");
if (index != -1) {
    System.out.println("Found at index: " + index);
} else {
    System.out.println("Not found");
}
```

Output:

Found at index: 14

3. contains()

- Checks if the specified substring is present in the string.
- Returns true if the substring is found, false otherwise.

```
String str = "Hello, world!";
if (str.contains("world")) {
    System.out.println("Substring found");
} else {
    System.out.println("Substring not found");
}
```

Output:

Substring found

4. startsWith() and endsWith()

- Checks if the string starts or ends with a specific substring.
- Returns true if the condition is met, false otherwise.

```
String str = "Hello, world!";
if (str.startsWith("Hello")) {
    System.out.println("Starts with 'Hello'");
}
if (str.endsWith("world!")) {
```

```
        System.out.println("Ends with 'world!'");
    }
```

Output:

Starts with 'Hello'

Ends with 'world!'

Example Program on searching methods of string:

```
class StringSearchingMethods {
    public static void main(String[] args) {
        String str = "Information Technology";

        // contains() method
        System.out.println("Contains 'Tech': " +
            str.contains("Tech")); // true

        // indexOf() method
        System.out.println("Index of 'o': " +
            str.indexOf('o')); // 3
        System.out.println("Index of 'o' from index 5: " +
            str.indexOf('o', 5)); // 9
        System.out.println("Index of 'form': " +
            str.indexOf("form")); // 2
        System.out.println("Index of 'tech': " +
            str.indexOf("tech")); // -1

        // lastIndexOf() method
        System.out.println("Last index of 'n': " +
            str.lastIndexOf('n')); // 16
        System.out.println("Last index of 'o' from index 5: " +
            str.lastIndexOf('o', 5)); // 3
        System.out.println("Last index of 'log': " +
            str.lastIndexOf("log")); // 18
        System.out.println("Last index of 'log': " +
            str.lastIndexOf("Log")); // -1

        // startsWith() method
        System.out.println("Starts with 'Inf': " +
            str.startsWith("Inf")); // true
        System.out.println("Starts with 'Techno' from index
            12: " + str.startsWith("Techno", 12)); // true

        // endsWith() method
        System.out.println("Ends with 'logy': " +
            str.endsWith("logy")); // true

        // matches() method
        String str1 = "Hello123";
        String str2 = "Hello@12";
```

```

        System.out.println("str1 matches regex [A-Za-z0-9]+: "
+ str1.matches("[A-Za-z0-9]+")); // true
        System.out.println("str2 matches regex [A-Za-z0-9]+: "
+ str2.matches("[A-Za-z0-9]+")); // false
    }
}

```

Output:

```

Contains 'Tech': true
Index of 'o': 3
Index of 'o' from index 5: 9
Index of 'form': 2
Index of 'tech': -1
Last index of 'n': 16
Last index of 'o' from index 5: 3
Last index of 'log': 18
Last index of 'log': -1
Starts with 'Inf': true
Starts with 'Techno' from index 12: true
Ends with 'logy': true
str1 matches regex [A-Za-z0-9]+: true

```

Topic: StringBuffer class in Java

- The `StringBuffer` class in Java is a part of the `java.lang` package and is used for creating and manipulating mutable sequences of characters.
- Unlike `String`, which is immutable (cannot be changed once created), `StringBuffer` allows you to modify the string it contains without creating new objects.

Common Methods

1. Constructors

- **`StringBuffer()`**: Creates an empty `StringBuffer` with an initial capacity of 16 characters.
- **`StringBuffer(String str)`**: Creates a `StringBuffer` initialized to the contents of the specified string.
- **`StringBuffer(int capacity)`**: Creates a `StringBuffer` with the specified initial capacity.

```

// Initial capacity 16
StringBuffer sb1 = new StringBuffer();

// Initial content "Hello"
StringBuffer sb2 = new StringBuffer("Hello");

// Initial capacity 50
StringBuffer sb3 = new StringBuffer(50);

```

2. Appending Data

- **append(String str)**: Appends the specified string to the end.
- **append(int i)**: Appends the string representation of the integer.
- **append(Object obj)**: Appends the string representation of the object.

```
StringBuffer sb = new StringBuffer("Hello");  
sb.append(" World"); // "Hello World"  
sb.append(2024); // "Hello World2024"
```

3. Inserting Data

- **insert(int offset, String str)**: Inserts the specified string at the specified position.
- **insert(int offset, int i)**: Inserts the string representation of the integer at the specified position.

```
StringBuffer sb = new StringBuffer("Hello World");  
sb.insert(6, "Beautiful "); // "Hello Beautiful World"
```

4. Deleting Data

- **delete(int start, int end)**: Removes the characters between the specified start and end indices.
- **deleteCharAt(int index)**: Removes the character at the specified position

```
StringBuffer sb = new StringBuffer("Hello Beautiful World");  
sb.delete(6, 16); // "Hello World"  
sb.deleteCharAt(5); // "HelloWorld"
```

5. Replacing Data

- **replace(int start, int end, String str)**: Replaces the characters between the specified start and end indices with the specified string.

```
StringBuffer sb = new StringBuffer("Hello World");  
sb.replace(6, 11, "Java"); // "Hello Java"
```

6. Reversing Data

- **reverse()**: Reverses the characters in the StringBuffer.

```
StringBuffer sb = new StringBuffer("Hello");  
sb.reverse(); // "olleH"
```

7. Capacity and Length

- **capacity()**: Returns the current capacity of the `StringBuffer`.
- **length()**: Returns the length (number of characters) of the `StringBuffer`.

```
StringBuffer sb = new StringBuffer("Hello");  
  
int length = sb.length(); // 5  
  
int capacity = sb.capacity(); // 21 (initial 16 plus any  
additional space)
```

8. Conversion

- **toString()**: Converts the `StringBuffer` to a `String`.

```
StringBuffer sb = new StringBuffer("Hello World");  
  
String str = sb.toString(); // "Hello World"
```

Topic: CharSequence Interface in java

- The `CharSequence` interface in Java is part of the `java.lang` package and represents a readable sequence of `char` values.
- It serves as a common interface for various classes that implement sequences of characters, such as `String`, `StringBuilder`, and `StringBuffer`.

Purpose of CharSequence

- The `CharSequence` interface provides a general way to access and manipulate character sequences without being tied to any specific implementation.
- This allows for greater flexibility when working with different types of character sequences.

Implementations of CharSequence

The `CharSequence` interface is implemented by several classes in Java:

- **String**: An immutable sequence of characters.
- **StringBuilder**: A mutable sequence of characters.
- **StringBuffer**: A mutable sequence of characters, similar to `StringBuilder`, but with synchronized methods for thread safety.

Methods

The `CharSequence` interface defines several methods that are used to interact with sequences of characters:

1. `charAt(int index)`

- Returns the character at the specified index.
- **Throws:** `IndexOutOfBoundsException` if the index is out of range.

```
CharSequence cs = "Hello, world!";  
char c = cs.charAt(7); // Returns 'w'
```

2. `length()`

- Returns the length of the character sequence.

```
CharSequence cs = "Hello, world!";  
int length = cs.length(); // Returns 13
```

3. `subSequence(int start, int end)`

- Returns a new `CharSequence` that is a subsequence of this sequence.
- **Throws:** `IndexOutOfBoundsException` if the start or end index is out of range.

```
CharSequence cs = "Hello, world!";  
CharSequence sub = cs.subSequence(7, 12); // Returns "world"
```

4. `toString()`

- Converts the `CharSequence` into a `String`. This method is not part of the `CharSequence` interface but is available on all classes that implement `CharSequence`.

Example:

```
class CharSequenceExample {  
    public static void main(String[] args) {  
        // Using CharSequence with String  
        CharSequence cs1 = "Hello, world!";  
        System.out.println(cs1.charAt(7)); // 'w'  
        System.out.println(cs1.subSequence(7, 12)); // "world"  
  
        // Using CharSequence with StringBuilder  
        CharSequence cs2 = new StringBuilder("Mutable  
sequence");  
        System.out.println(cs2.charAt(0)); // 'M'  
        System.out.println(cs2.subSequence(0, 7)); //  
"Mutable"  
  
        // Using CharSequence with StringBuffer  
        CharSequence cs3 = new StringBuffer("Another mutable  
sequence");  
        System.out.println(cs3.charAt(8)); // 'm'
```

```
        System.out.println(cs3.subSequence(8, 15)); //  
"mutable"  
    }  
}
```

Output:

```
w  
world  
M  
Mutable  
m  
mutable
```